

```
In [1]: import pandas as pd

# Load the CSV file (same folder as notebook)
df = pd.read_csv("Vacant_Properties.csv")

# Basic checks
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

# Preview data
df.head()
```

Dataset shape: (1651, 17)

Column names:
['X', 'Y', 'SBL', 'PropertyAddress', 'Zip', 'Owner', 'OwnerAddress', 'Vacant', 'neighborhood', 'VPR_result', 'completion_date', 'completion_type_name', 'valid_until', 'VPR_valid', 'Latitude', 'Longitude', 'ObjectId']

Out[1]:

	X	Y	SBL	PropertyAddress	Zip	Owner	OwnerAddress
0	NaN	NaN	002.-04-02.4	1920 Park St	13208	Wellington Ward LLC	100 Windsor Syracuse, NY 13213
1	-8.476690e+06	5.315382e+06	077.-23-25.0	316 Warner Ave	13205	Donna Larode	321 Warn Ave Syracuse NY 13204
2	-8.476078e+06	5.315534e+06	077.-10-15.0	2223 State St S & Amherst Ave	13205	Vladimir Dyomin	2644 Torr Pines Rd Jolla, CA 92037
3	-8.479473e+06	5.319570e+06	109.-05-04.0	353 Richmond Ave	13204	Jason Yagan	353 Richmor Ave Syracuse NY 13204
4	-8.475432e+06	5.320042e+06	030.-03-01.0	500 Hawley Ave & Crouse Ave N	13203	City of Syracuse	233 Washington Syracuse, NY 13203

```
In [2]: df.isna().sum().sort_values(ascending=False)
```

```
Out[2]: valid_until      1276
        VPR_result      1276
        completion_type_name 1276
        completion_date   1276
        VPR_valid         1276
        X                 36
        Y                 36
        Latitude          36
        Longitude         36
        neighborhood      15
        OwnerAddress       0
        Vacant             0
        Zip               0
        SBL               0
        PropertyAddress    0
        Owner              0
        ObjectId           0
        dtype: int64
```

```
In [3]: (df.isna().mean() * 100).round(2).sort_values(ascending=False)
```

```
Out[3]: valid_until      77.29
        VPR_result      77.29
        completion_type_name 77.29
        completion_date   77.29
        VPR_valid         77.29
        X                 2.18
        Y                 2.18
        Latitude          2.18
        Longitude         2.18
        neighborhood      0.91
        OwnerAddress       0.00
        Vacant             0.00
        Zip               0.00
        SBL               0.00
        PropertyAddress    0.00
        Owner              0.00
        ObjectId           0.00
        dtype: float64
```

```
In [4]: null_summary = pd.DataFrame({
        "Missing_Count": df.isna().sum(),
        "Missing_Percent": (df.isna().mean() * 100).round(2)
    }).sort_values("Missing_Count", ascending=False)

null_summary
```

Out[4]:

	Missing_Count	Missing_Percent
valid_until	1276	77.29
VPR_result	1276	77.29
completion_type_name	1276	77.29
completion_date	1276	77.29
VPR_valid	1276	77.29
X	36	2.18
Y	36	2.18
Latitude	36	2.18
Longitude	36	2.18
neighborhood	15	0.91
OwnerAddress	0	0.00
Vacant	0	0.00
Zip	0	0.00
SBL	0	0.00
PropertyAddress	0	0.00
Owner	0	0.00
ObjectId	0	0.00

```
In [5]: data_dict = pd.DataFrame({
    "Column": df.columns,
    "Data Type": df.dtypes,
    "Missing Values": df.isna().sum()
})

data_dict
```

Out[5]:

	Column	Data Type	Missing Values
X	X	float64	36
Y	Y	float64	36
SBL	SBL	object	0
PropertyAddress	PropertyAddress	object	0
Zip	Zip	int64	0
Owner	Owner	object	0
OwnerAddress	OwnerAddress	object	0
Vacant	Vacant	object	0
neighborhood	neighborhood	object	15
VPR_result	VPR_result	object	1276
completion_date	completion_date	object	1276
completion_type_name	completion_type_name	object	1276
valid_until	valid_until	object	1276
VPR_valid	VPR_valid	object	1276
Latitude	Latitude	float64	36
Longitude	Longitude	float64	36
ObjectId	ObjectId	int64	0

```
In [6]: analysis_df = df.drop(columns=[
    "VPR_result",
    "VPR_valid",
    "completion_date",
    "completion_type_name",
    "valid_until"
])
```

```
In [7]: geo_df = analysis_df.dropna(subset=["Latitude", "Longitude"])
```

```
In [8]: count_df = analysis_df.copy()
```

```
In [9]: neighborhood_counts = count_df["neighborhood"].value_counts()
neighborhood_counts.head(10)
```

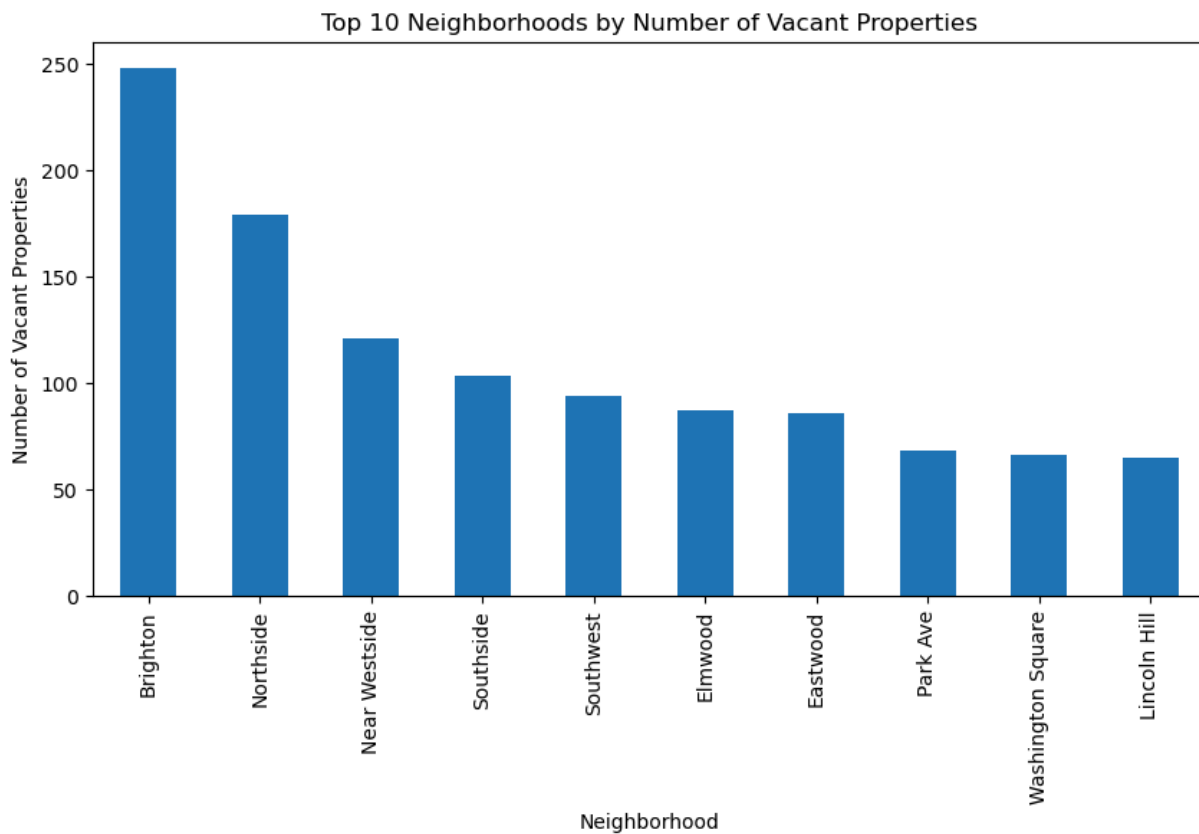
```
Out[9]: neighborhood
        Brighton      248
        Northside     179
        Near Westside  121
        Southside     103
        Southwest      94
        Elmwood        87
        Eastwood       86
        Park Ave       68
        Washington Square 66
        Lincoln Hill   65
        Name: count, dtype: int64
```

```
In [10]: zip_counts = count_df["Zip"].value_counts()
        zip_counts.head(10)
```

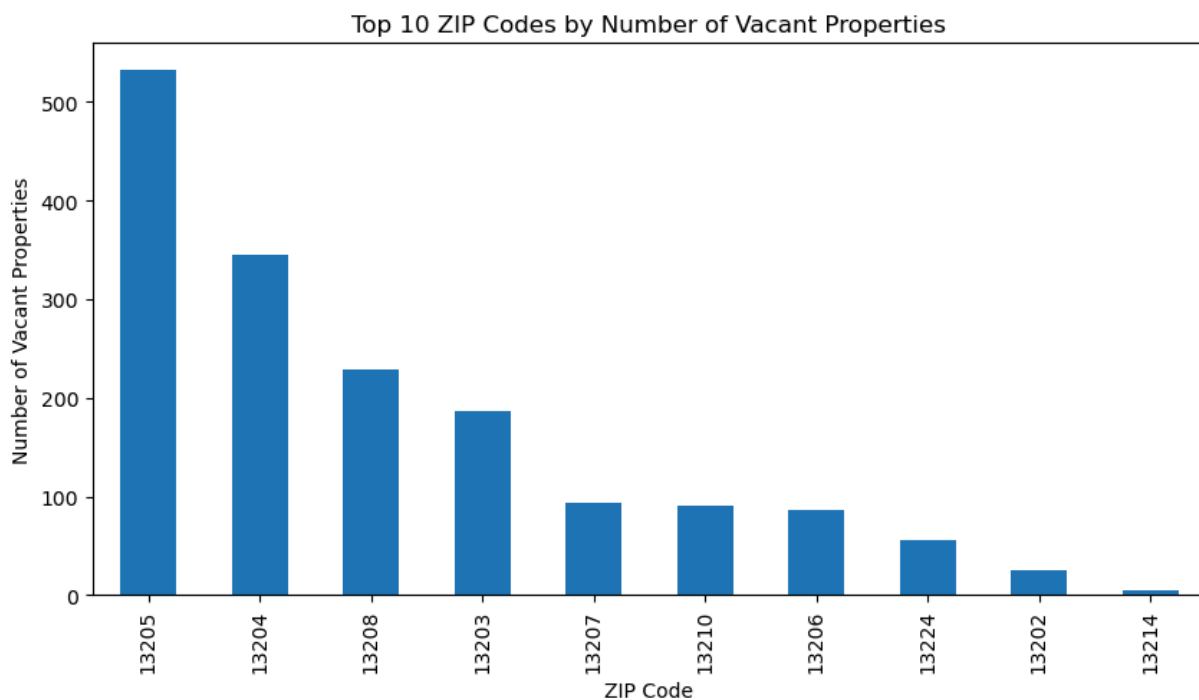
```
Out[10]: Zip
        13205      533
        13204      345
        13208      228
        13203      186
        13207       94
        13210       91
        13206       86
        13224       56
        13202       25
        13214        5
        Name: count, dtype: int64
```

```
In [11]: import matplotlib.pyplot as plt

        neighborhood_counts.head(10).plot(kind="bar", figsize=(10,5))
        plt.title("Top 10 Neighborhoods by Number of Vacant Properties")
        plt.xlabel("Neighborhood")
        plt.ylabel("Number of Vacant Properties")
        plt.show()
```

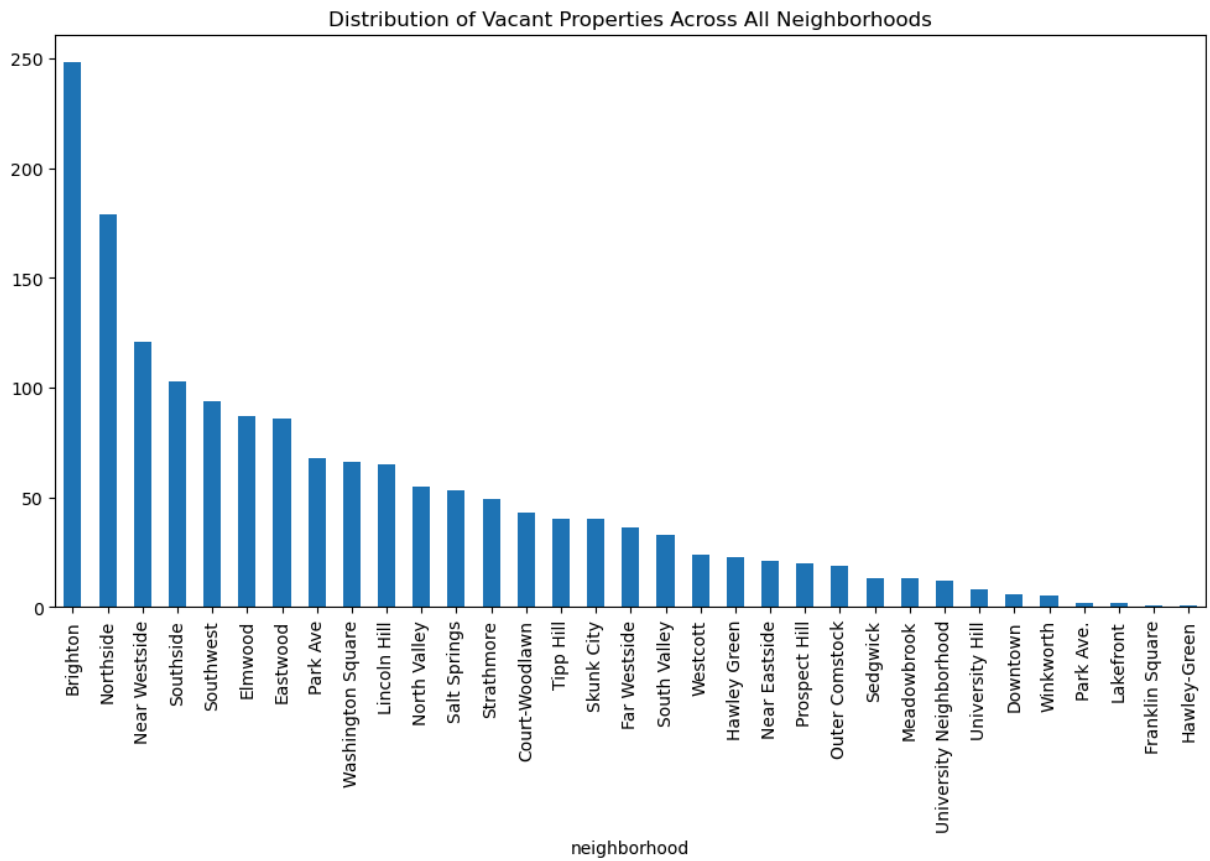


```
In [12]: zip_counts.head(10).plot(kind="bar", figsize=(10,5))
plt.title("Top 10 ZIP Codes by Number of Vacant Properties")
plt.xlabel("ZIP Code")
plt.ylabel("Number of Vacant Properties")
plt.show()
```



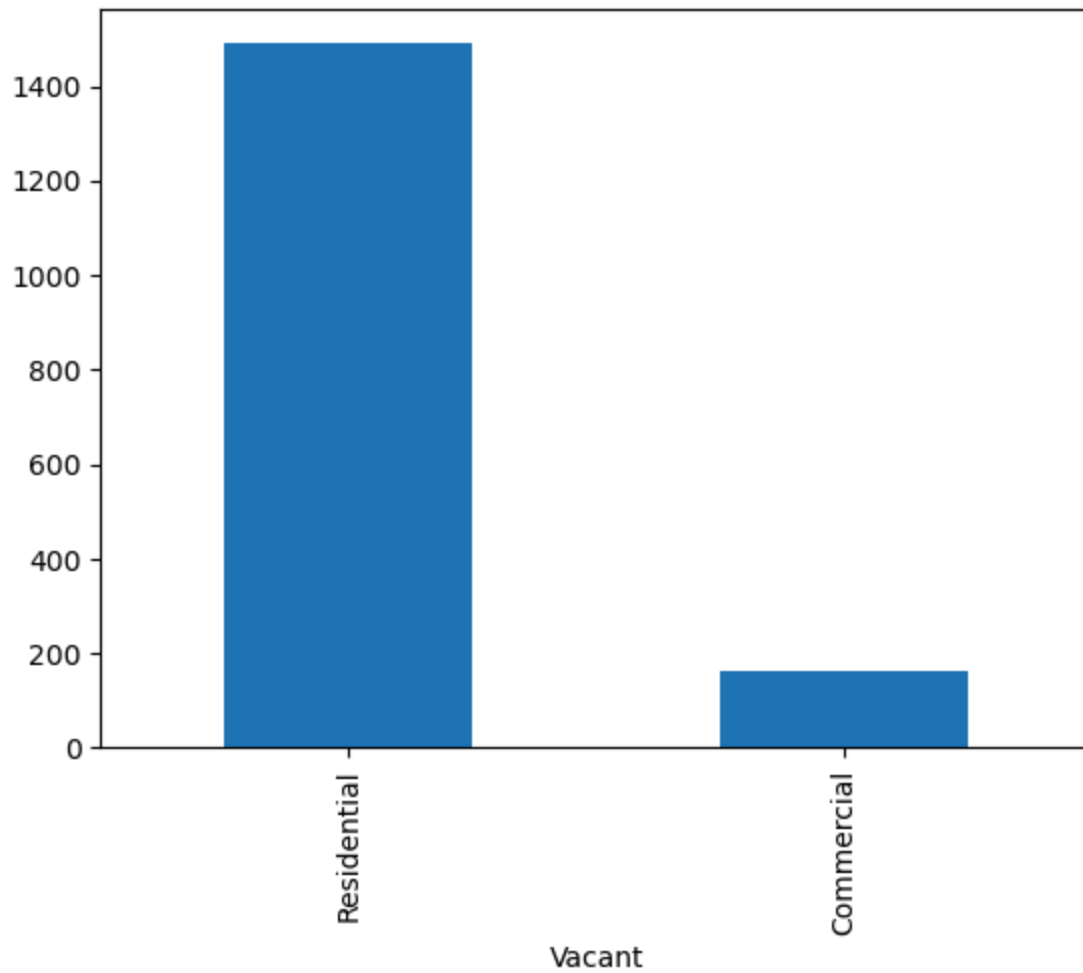
```
In [13]: neighborhood_counts.plot(kind="bar", figsize=(12,6))
plt.title("Distribution of Vacant Properties Across All Neighborhoods")
```

```
plt.show()
```

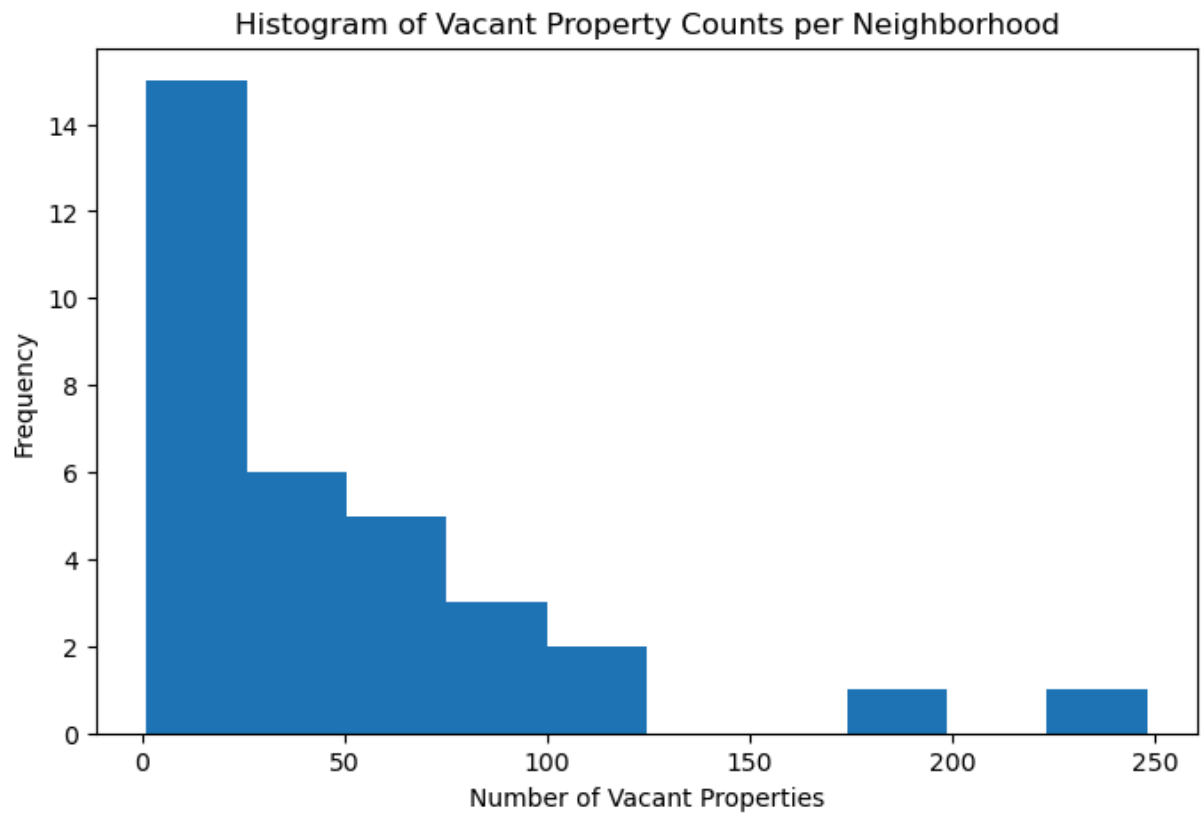


```
In [14]: count_df["Vacant"].value_counts().plot(kind="bar")
```

```
Out[14]: <Axes: xlabel='Vacant'>
```

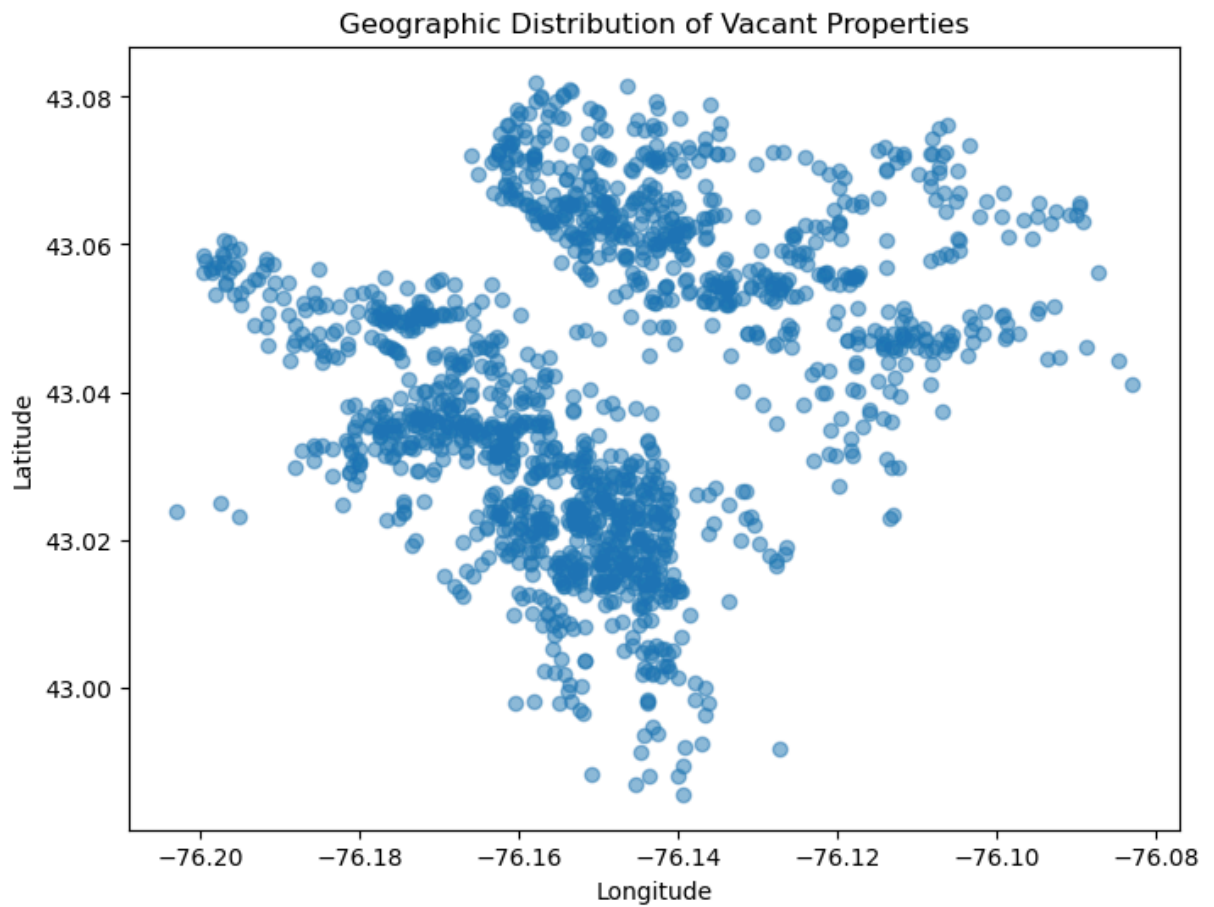


```
In [15]: plt.figure(figsize=(8,5))
plt.hist(neighborhood_counts, bins=10)
plt.title("Histogram of Vacant Property Counts per Neighborhood")
plt.xlabel("Number of Vacant Properties")
plt.ylabel("Frequency")
plt.show()
```

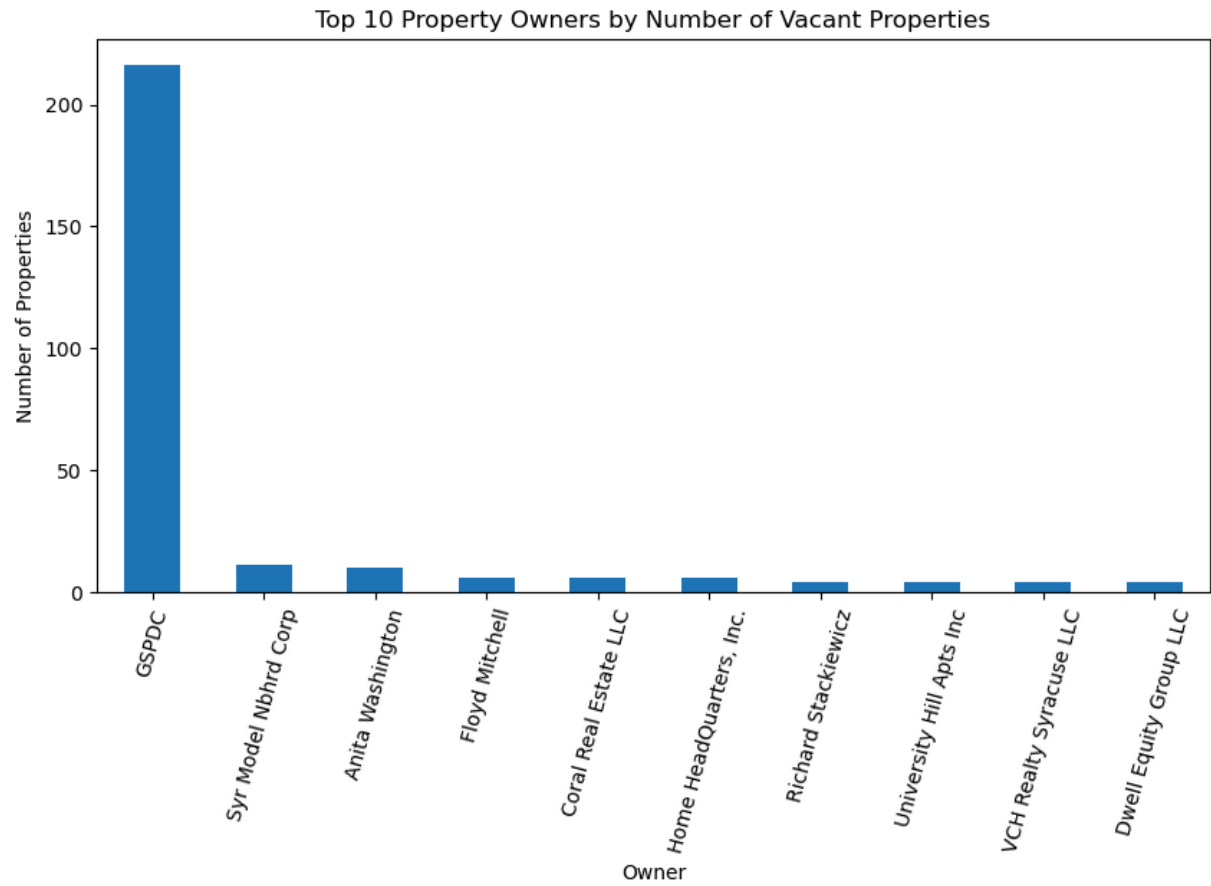
```
In [16]: geo_df = analysis_df.dropna(subset=["Latitude", "Longitude"])

plt.figure(figsize=(8,6))
plt.scatter(geo_df["Longitude"], geo_df["Latitude"], alpha=0.5)
plt.title("Geographic Distribution of Vacant Properties")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.show()
```



```
In [18]: top_owners = count_df["Owner"].value_counts().head(10)

plt.figure(figsize=(10,5))
top_owners.plot(kind="bar")
plt.title("Top 10 Property Owners by Number of Vacant Properties")
plt.xlabel("Owner")
plt.ylabel("Number of Properties")
plt.xticks(rotation=75)
plt.show()
```



```
In [ ]:
```

```
In [ ]:
```